Lessons Learned: AICSEPAR Workshop on GenAI in CS Education

Author: Eric Ringger, BYU Computer Science (aided by Claude AI)

Contact: ringger@cs.byu.edu

Audience: BYU CS Department Faculty (and shared more broadly to relevant audiences)

Source: GenAI in CS Education Workshop: Practice and Research (AICSEPAR), March 16-17, 2026, La Jolla, CA. Supplemented by an informal session with Cory Gwin (GitHub Education) and review of Karpathy's autoresearch project.

Companion document: AICSEPAR-Workshop-Notes-with-Slides (full session notes, slide references, people, follow-ups, and annotated resources) – warning: long!

Last updated: March 19, 2026, 8:30 AM MDT

Status: Working draft

Key Principles for Department Discussion

Primary focus: undergraduate CS education. Principle 10 extends to graduate training and research.

Organized around the student: who they are, what they need to learn, how we teach and assess, and the institutional context.

THE WORLD HAS CHANGED, AND WE GET TO ADAPT NOW

> "Materials for my course are already out of date. Goals are still largely appropriate." — Bill Pugh (UMD), on an AI-first course launched one semester earlier — the tools change fast, but the right learning goals hold

> "I no longer write or read code. I've been coding for 53 years." — Bill Pugh

> "Last 3 months: haven't written a single line of code myself." — Cory Gwin (GitHub Education)

WHO ARE OUR STUDENTS?

1. Student goals matter, and we may be teaching two different populations

> "Mastery-oriented students seek more instrumental over executive help." — Stephanie Yang (Harvard)

> "People have been pushing forever: everyone should code. Now everyone can code." — Cory Gwin (GitHub Education)

> "88.9% of students used gen AI tools prior to the course, learned only through informal means." — Gorson Benario et al. (Google)

Students don't experience AI tools neutrally. A student who wants to learn uses AI differently than a student who wants to finish. If we don't understand our students' goals, we can't design courses where AI helps rather than hinders; and the same assignment will develop one student while enabling another to coast.

Stephanie Yang's Harvard study made this concrete: mastery-oriented students seek instrumental help from LLMs (guidance to deepen understanding), while performance-oriented and work-avoidant students seek executive help (quick answers to finish tasks). Her finding that "perception of appropriateness matters" means that what students *believe* about proper AI use (shaped by course norms, peer culture, and their own goals) directly determines whether the AI becomes a learning tool or a shortcut.

This connects to Cory Gwin's observation from the industry side: CS is splitting into two populations. One cohort does deep technical work: system design, architecture, security, the kind of expertise that requires sustained formation. The other uses AI to build what they need (Gwin's wife built a museum event-tracking tool with Claude Code, and his mom built an app). Both are legitimate. But they require fundamentally different educational experiences, and "can't teach both in the same way."

The Michigan GenAI contracts approach this from the student side by asking each student to articulate their own goals and set conditional rules for AI use *before* the work begins. 58% said the contract changed their thinking; but the mechanism is goal-awareness, not restriction. Students who know what they're trying to learn make better decisions about when AI helps and when it hinders.

The Gorson Benario (Google) study adds a temporal dimension: 88.9% of students were already using GenAI before any formal instruction, having learned through informal means only. Their habits and goals were already formed. Formal instruction changed behavior; but the study was conducted with mid-level students at minority-serving institutions in early 2024. The authors acknowledged it needs to be re-run in the age of agentic coding, where the gap between casual use and expert use is much wider.

The curriculum implication: We need to know what our students' goals are, and design accordingly. A mastery-oriented student in a systems course needs different AI scaffolding than a business major taking CS1 to "build things." The Yang research suggests we should be

measuring student motivation and goal orientation, not just outcomes. The Gwin split suggests we may need explicitly differentiated tracks, or at least explicitly differentiated learning contracts within the same course.

The open question for our department: which population are we primarily serving, and does our curriculum reflect that? Are we preparing deep technical practitioners, broad AI-augmented builders, or both, and if both, are we honest about the different paths?

Key references:

- Yang et al. (https://dl.acm.org/doi/10.1145/3769994.3770009)
- Gwin industry panel: two personas for CS (see notes, Industry Panel section)
- Michigan GenAI Contracts (Aadarsh Padiyath, U. Michigan)
- Gorson Benario et al. (https://dl.acm.org/doi/10.1145/3641554.3701859)

2. Student agency is the prize, and the risk

> "Accelerating experimentation → Better learning → More agency." — Lunch discussion note
>
> "Teaching our students Agency! How, when, and why to use GenAI tools." — Claudia Szabo (Adelaide)
>
> "Just ChatGPT that." — What students say to each other instead of helping (Temple study)

Agency stands out as both the biggest opportunity and the biggest danger of GenAI in education (see also Elder Bednar, "Things as They Really Are, 2.0"). The concern is real: students surrender agency by defaulting to AI for decisions they should be making themselves. The Temple study's "just ChatGPT that" captures the problem: students replacing peer help-seeking and independent problem-solving with reflexive tool use. Kevin Lin found students won't even read what the AI produces. If the tool does the thinking, the student doesn't develop judgment.

But the workshop also surfaced the other side: AI can *return* agency to students by removing bottlenecks that previously constrained what they could attempt. This insight emerged during the Monday lunch discussion ("accelerating experimentation → better learning → **more agency**") and was validated when I posed the question directly to Titus Winters. Winters' response: junior engineers have been stuck in box-checking and rule-following, and AI frees them to "invest more time and energy required for running experiments, thinking critically." That still requires effort and judgment; but now students can try more things, fail faster, and iterate on ideas that were previously out of reach in a semester.

The positive agency case across the workshop:

- **Szabo** (Adelaide) made agency a first-class learning goal: "Teaching our students Agency! How, when, and why to use GenAI tools." Her incremental curriculum is explicitly designed to build student judgment about *when* to use AI and when not to; agency as a skill, not a disposition.
- **Pugh** (UMD) saw projects shrink from "2 weeks to 2 hours": which doesn't mean less learning; it means students can attempt more ambitious projects and iterate on them. His course has students building full social media apps and tackling legacy codebases in contexts that would have been impossible in a single semester before.
- **Dodds** (Harvey Mudd) found that DCbA assessment "improves agency and accountability": when students must demonstrate and explain their work live, they own it in a way that submitting code to an autograder never required.
- **Karpathy's autoresearch** is the research version: one researcher with one GPU running 100 experiments overnight. The agency shifts from "can I run this experiment?" to "which experiment is worth running?" (a higher-order form of agency).
- **Denny's probeable problems** give students agency over problem clarification: they decide what to probe, when, and how many times. Higher-performing students exercise more of this agency.

The surrendered agency case:

- **Temple** ("just ChatGPT that"): 100% of students use AI; social learning communities eroding. The tool becomes a substitute for engagement, not a catalyst for it.
- **Lin** (UW): Students avoid reading AI-generated code. They prompt and iterate without engaging with the output: agency over the *process* without agency over the *understanding*.
- **Michigan GenAI contracts**: 58% of students said the contract changed their thinking about AI use; which means 42% didn't. Some students under deadline pressure abandon their own guidelines. Agency requires scaffolding, not just exhortation.

The design challenge: The difference between gained agency and surrendered agency comes down to course design. Problem-solving-first assignments (Principle 5) force students to exercise judgment. Reading-first pedagogies (Principle 4) force students to engage with output. GenAI contracts (Principle 1) force students to articulate their own goals. Assessment that requires demonstration (Principle 6) makes it impossible to hide behind the tool. None of these happen by default; they require deliberate design choices by the instructor.

The open question: can we design courses where AI consistently *amplifies* student agency rather than replacing it? The workshop suggests the answer is yes, but only if we treat agency as a learning outcome to be designed for, not an automatic byproduct of giving students better tools.

Key references:

- Winters Q&A: freeing students from box-checking (see notes, Winters keynote Q&A)
- Szabo: agency as a first-class learning goal (see notes, Szabo session)

- Pugh course: projects shrink from 2 weeks to 2 hours (see notes, Pugh session)
- Dodds DCbA: agency and accountability (see notes, Dodds session)
- Denny probeable problems: agency over clarification (see notes, Denny session)
- Temple study: surrendered agency, "just ChatGPT that" (see notes, Temple session)
- Lin slides: students avoided reading AI code (see notes, UW lightning talk)
- Michigan GenAI Contracts
- Karpathy autoresearch: discussed in Gwin agentic coding session (see notes, GitHub Agentic Coding section)

WHAT DO STUDENTS NEED TO LEARN?

> "AI-enhanced developer displacing the developer who does not reskill and upskill to keep up with AI tools." — Kam & Malyn-Smith (Google)
>
> "Entry-level is now expected to do what senior-level engineers used to be expected to do." — Kam & Malyn-Smith
>
> "Thinking is more important than coding." — Artur Borycki (Teradata)

3. Design and design thinking are the new core, in every course

> "Coding is not the bottleneck, and never will be again." — Titus Winters (Adobe)
>
> "Design education and testing education ARE AI education." — Titus Winters
>
> "We all pretend that this is applied math. This is not a math problem. It's a craft." — Titus Winters
>
> "Design, specify, architect, prompt." — Titus Winters

Students entering the workforce will spend their time on system design, specification, testing, and evaluation, not writing much code. If we don't teach design thinking as a pervasive skill, we are preparing students for a role that is rapidly shrinking while neglecting the role that is expanding.

Titus Winters' thesis ("design education and testing education are AI education") was the most quotable articulation. The design thread ran through nearly every session. When AI handles code generation, what remains is design: system design, specification design, test design, prompt design, evaluation design. This isn't a new elective to add; it's a discipline-wide shift in what "doing CS" means.

The scope of "design" that emerged from the workshop:

- **System design and architecture:** The Kam/Malyn-Smith occupational profile found that AI-enhanced developers spend *more* time on system design and technical decision-making, not less. The SDLC mapping shows that most of the 75 professional

tasks are in Plan, Test, Deploy, Operate, and Monitor (not in Code). Kevin Lin's UW course was explicitly organized around principles of system design, decomposition, and verification (drawn from Google DORA 2025).

- **Specification as design:** Shapiro framed specification as "the articulation of human intent": a design activity, not a clerical one. His four contradictions show why: when programs are hybrid (code + models + data + prompts), specifying what you want is harder and more consequential than implementing it. Winters reinforced this: "the prompt alone is not enough."
- **Test design:** Shapiro's envisioned future IDEs generate cross-validation test plans for statistically-defined functions. Winters argued that knowing *what quality means* and *how to verify it* is the load-bearing expertise. Denny's probeable problems are fundamentally exercises in test design: students must figure out what to probe and why.
- **Interaction design:** Shapiro's HAISE vision is about designing human-AI collaboration patterns. Gwin described "building the system that's building the system" (designing agentic architectures, not writing code). The Kam/Malyn-Smith IAVES framework (Initiate, Analyze, Verify, Elevate, Socialize) is itself a design pattern for human-AI interaction.
- **Evaluation design:** Baker Franke's shift from specification to solution analysis is a design problem. Yang's research on chatbot help-seeking is a design problem. The agentic first-pass evaluation model (Principle 8) requires designing what the agent assesses and how the TA reviews it.

The curriculum implication: Design thinking (in the broad sense of decomposition, specification, tradeoff analysis, and evaluation) belongs in every CS course, not just a software engineering elective. CS1 needs design (Winters: "bring more design and testing into the course from the beginning"). Data structures needs design (how do you specify what a correct solution looks like when there are multiple valid approaches?). Systems courses need design (Gwin's agentic architectures). Capstone courses need design (the CMU model of building things that actually ship).

Winters put it most sharply: "We all pretend that this is applied math. This is not a math problem. It's a craft." If we accept that, then design is the craft skill, and it needs to permeate the curriculum the way mathematical rigor currently does.

Key references:

- Winters keynote (see notes, Winters session)
- Shapiro keynote: HAISE framework, four contradictions (see notes, Shapiro session)
- Kam & Malyn-Smith paper (https://arxiv.org/abs/2506.00202) and IAVES framework
- Lin slides: system design principles, DORA 2025 (see notes, UW lightning talk)
- Denny: CodeRunner (https://coderunner.org.nz)
- CMU AI Tools course (https://ai-developer-tools.github.io/)

4. Students must learn to read and supervise code, not just write it

> "Students avoided reading AI-generated code." — Kevin Lin (UW)

> "You can drown in generated code. 75% more PRs coming into GitHub now than 3 months ago." — Cory Gwin (GitHub Education)

Our students will spend their careers reading, evaluating, and reviewing code they didn't write (generated by AI agents, written by teammates, inherited from legacy systems). Yet reading code is the skill students are least inclined to practice. If we don't deliberately develop reading fluency, we are sending graduates into a profession where the primary daily activity is something they've been trained to avoid.

Shapiro made reading foundational by adding "Read Code" as a first-class element of programming practice in his updated coin model. The Porter-Zingaro "Read, Critique, & Improve" framework puts reading *first* in the pedagogical sequence: students encounter and critique code before they produce it. Shapiro's four contradictions provide a structured argument for why the old write-first pedagogy is broken when programs are "hybrid" (code + models + data + prompts). Shapiro's updated coin model adds "Evaluate GenAI output quality" and "Verify conformance to specification" as essential SE practices.

The industry panel confirmed that reading is already the daily work. Cory Gwin's (GitHub) daily work has shifted entirely to reading: reviewing agent traces, assessing risk in generated code, debugging multi-agent orchestration. In a world with 75% more PRs flowing through GitHub than three months ago, the ability to read quickly and judge accurately is becoming the bottleneck skill. Magaly Drant (ServiceNow) asked whether engineers will "start reviewing code all day"; the answer is increasingly yes. Artur Borycki (Teradata) now asks interviewees to write system prompts rather than code.

Yet Kevin Lin's UW course found that students *avoid* reading AI-generated code. They'll prompt, they'll iterate, but they won't read what came back. This is perhaps the most actionable gap in the entire workshop: the skill industry needs most is the one students are least inclined to practice. The Illinois LLM tutoring study tackled this directly, using chatbots to walk students through code line by line, breaking the habit of skimming. Baker Franke's student who said "it was so much faster to write the code myself" reveals the deeper tension: reading someone else's code (or an AI's code) feels harder and less satisfying than writing your own.

The curriculum implication: We need assignments where reading is the *primary* activity, not a byproduct. Several workshop models point the way: The Porter-Zingaro "Read, Critique, & Improve" sequence structures reading as the entry point. Chris Murphy's OO Design course at Columbia has students critically assess software artifacts across six assignments. Pugh's later course modules (Congressional Record codebase, legacy office hours system) put students into unfamiliar code they didn't write. The open question: can we build a curriculum where students develop reading fluency as deliberately as we currently develop writing fluency, and will they tolerate it?

Key references:

- Shapiro keynote: updated coin model with "Read Code" (see notes, Shapiro session)
- Porter-Zingaro book (https://www.manning.com/books/learn-ai-assisted-python-programming-second-edition)
- Lin slides: students avoided reading AI code (see notes, UW lightning talk)
- Kam & Malyn-Smith paper (https://arxiv.org/abs/2506.00202)
- Illinois/PrairieLearn: chatbot-guided code reading (https://www.prairielearn.com/)
- Murphy, Columbia: OO Design, 6 assignments critically assessing artifacts (see notes, Murphy session)
- Pugh course: brownfield coding modules: Congressional Record, legacy office hours system (see notes, Pugh session)

5. Problem-solving is the skill; ambiguity is the condition that develops it

> "When was the last time you encountered a problem on the job for which correctness was well defined?" — Titus Winters (Adobe)

> "I find these tasks often quite difficult because of how vague the question is. This means that I have to think ahead of time to work out what I am supposed to do instead of writing code." — A student working on Denny's probeable problems

AI solves fully-specified problems. The clearer the assignment, the less the student needs to think. This is the new reality. The question is not how to make assignments AI-proof; it's how to develop students who can solve problems that don't come pre-specified, because those are the problems that matter in practice.

The reality: Well-defined problems (build Asteroids; it exists in training data) are solved. AI retrieves, elaborates, and generates. Ill-defined problems (the ones that require contact with reality, domain judgment, and sustained engagement with people) remain human. You cannot prompt-engineer your way to a good problem statement. Shapiro: "specification is the articulation of human intent." Winters: "the prompt alone is not enough." The industry panel consensus: the real skill is understanding requirements and working with business people. If we only give students well-defined assignments, we are training them for exactly the work that AI already does.

The pedagogical insight: Multiple speakers independently arrived at the same conclusion: intentional ambiguity in assignments is what creates the space for genuine problem-solving. Schneider argued this in 1978. Paul Denny built it into a scalable system: his probeable problems strip details from specifications and give students an oracle to query. Students must clarify requirements, identify edge cases, and reason about what's being asked *before* they write any code. His data shows this works: higher-performing students probe more, and students report learning to "think ahead of time to work out what I am supposed to do instead of writing code."

In CS1: Baker Franke designed projects around spatial reasoning and ambiguous specifications (not to frustrate students, but because these are the problems that force genuine

problem-solving). ChatGPT doesn't get Karel the Robot right off the bat. His design principles: spatial reasoning, ambiguity, and code integration from multiple students all require students to think, not just prompt. The key shift for instructors: less effort on writing tight specifications, more effort on analyzing how students solve problems.

In industry: Titus Winters challenged the correctness-first orientation of CS education: "When was the last time you encountered a problem on the job for which correctness was well defined?" The Kam/Malyn-Smith occupational profile confirms it: AI-enhanced developers spend more time on system design and technical decision-making (which is problem-solving) than on code production.

The open question: Can problem-solving judgment develop without prior generative experience? Baker Franke's student said "it was so much faster to write the code myself, and then I understood it." The foundation may still matter; but as a basis for supervisory judgment, not production.

Practical models from the workshop:

- **Probeable problems** (Denny): Students probe an oracle to clarify an intentionally ambiguous spec: the problem-solving happens *before* any code is written. Scales via CodeRunner.
- **Spatial/ambiguous projects** (Franke): Karel the Robot, pathfinding challenges, game design (problems that require students to decompose and reason, not just prompt).
- **Brownfield coding** (Pugh): Drop students into unfamiliar legacy codebases (Congressional Record parser, office hours queue system): problem-solving in the real-world sense of understanding someone else's decisions.
- **GenAI Contracts** (Padiyath, Michigan): Students define their own problem-solving boundaries for AI use: metacognitive engagement with when and how to seek help.

Key references:

- Shapiro keynote: specification as "articulation of human intent" (see notes, Shapiro session)
- Winters keynote: "the prompt alone is not enough" (see notes, Winters session)
- Denny: probeable problems and CodeRunner (https://coderunner.org.nz)
- Franke: spatial/ambiguous projects and break conversation (see notes, Franke session)
- Pugh course: brownfield coding modules (see notes, Pugh session)
- Michigan GenAI Contracts

HOW DO WE TEACH AND ASSESS?

6. We must assess what actually matters, and there are working models

> "CS1's goal is creating skills, not artifacts." — Zach Dodds (Harvey Mudd College)

If we say that problem-solving, design thinking, code reading, and supervisory judgment are what students need to learn (Principles 3-5), then those are what we need to assess. Assessing code artifacts that AI can produce is no longer a meaningful signal of student learning. The workshop presented several concrete models that assess the skills we claim to value:

Probeable problems (Denny, Auckland): Make assignments intentionally ambiguous. Students probe an oracle to discover edge cases before coding. Higher-performing students probe more; lower-performers jump to code. Scales via CodeRunner.

DCbA (Demonstration + Conversation-based Assessment) (Dodds, Harvey Mudd/Caltech): Weekly demos where students demonstrate, explain ("Understand"), and experiment. TAs follow a script. Works regardless of faculty stance on LLMs. CS1's goal is skills, not artifacts.

GenAI Contracts (Aadarsh Padiyath, U. Michigan): Students create personal contracts with goals, conditional AI usage rules, and midterm reflections. 58% reported the contract changed their GenAI thinking. Low-effort to implement, applicable across all CS courses.

LLM-integrated assessments (Hassan/Denny/Zilles, Illinois): Chatbots inside PrairieLearn that guide students through problem-solving steps without giving away answers. Effective as "process instructors" in a secure testing facility.

Key references:

- Denny: probeable problems (see notes, Denny session; https://coderunner.org.nz)
- Dodds: DCbA model (see notes, Dodds session)
- Michigan GenAI Contracts lightning talk
- PrairieLearn: LLM-integrated assessments (https://www.prairielearn.com/)

7. GenAI should be "peanut-buttered" across the curriculum, not bolted on

> "Don't teach AI tools in isolation." — Kam & Malyn-Smith (Google)
>
> "Course coordinators have made the time — 'on the adventure with me.'" — Claudia Szabo (Adelaide), on bringing faculty along

Students need coherent, progressive development of AI skills across their entire program (not a single course that teaches "the AI stuff" in isolation). A student who encounters GenAI only in a standalone elective won't develop the judgment to use it wisely in software engineering, data structures, or systems. The skills from Principles 3-5 (design, reading, problem-solving) must be practiced with AI tools throughout the curriculum, not just once.

Claudia Szabo's Adelaide model is the most complete example: a three-year progression from basic tools and prompting (Year 1) through decomposition and integration (Year 2) to full agentic AI project management (Year 3). Built from scratch during a university merger; accredited by two bodies.

Bill Pugh's standalone 1-credit course is the quick-start alternative: ready-made materials, open-ended projects, all coding in class. The Kam/Malyn-Smith summary recommendation: "Don't teach AI tools in isolation. Teach students to use AI tools integrated throughout the development lifecycle within human-in-the-loop workflows."

The UW lightning talk (Kevin Lin) organized around Google DORA 2025 principles: system design, verification and accountability, code review. Students avoided reading AI-generated code; 2 credits was insufficient; explicit guidance on modularization was needed.

Key references:

- Szabo: three-year curricular roadmap (see notes, Szabo session)
- Pugh course materials (https://github.com/billpugh/cmsc398z-student-downloads)
- Lin: UW course lessons learned (see notes, UW lightning talk)

8. Faculty effort shifts from specification to evaluation, and agentic tools can help it scale

> "Less: specification. More: solution analysis." — paraphrasing Baker Franke (UIC), in conversation at the break

Students doing open-ended, problem-solving-centered work (Principles 3-5) need rich, substantive feedback (not just a score from an autograder checking against a reference solution). If we ask students to design, read, and solve ill-defined problems but then assess them with tools built for well-defined problems, we undermine the learning we claim to value. The bottleneck is faculty and TA capacity to evaluate this kind of work.

This has a scaling problem. Evaluating open-ended student work (where there is no single correct answer) is expensive. Several workshop models already address pieces of this:

- **Dodds' DCbA** (Harvey Mudd/Caltech): TAs conduct scripted live conversations with students. 24 TAs at 2 hours/week, with 4 TAs handling 40 students per session. It works, but it's labor-intensive and depends on a premier TA corps.
- **Denny's oracle** (Auckland): The autograder still exists, but the pedagogically valuable work happens *before* submission, in the probing phase (which is self-service and scales without additional human effort).
- **Illinois LLM tutoring** (PrairieLearn): A chatbot provides first-pass process guidance during problem-solving, walking students through steps without giving away answers. The AI handles the high-volume formative interaction; human assessment focuses on summative evaluation.

The natural extension (not explicitly proposed at the workshop but implied by the pattern) is agentic first-pass evaluation of student work, followed by TA or faculty review of the AI's assessment. This mirrors what industry is already doing: Cory Gwin described building agents that evaluate other agents' outputs. The same architecture applies to teaching: an AI agent reads student code and produces a structured first-pass evaluation (identifying issues, noting strengths, flagging ambiguities in the student's approach), and a TA reviews that evaluation

rather than reading every submission from scratch. The TA's role shifts from grader to evaluator-of-evaluations (the same supervisory judgment shift that industry is experiencing).

This could address the "how to scale" question that came up repeatedly in Discussion sections (Dodds, Illinois, Szabo). The key design constraint: the agentic evaluation must be structured to *support* TA judgment, not replace it. TAs who rubber-stamp AI assessments learn nothing; TAs who critically review AI assessments develop exactly the supervisory skills the industry panel described.

Key references:

- Franke: "less specification, more solution analysis" (see notes, break conversation)
- Dodds: DCbA TA scaling model (see notes, Dodds session)
- Illinois/PrairieLearn: LLM process guidance (https://www.prairielearn.com/)
- Denny: oracle as self-service probing (see notes, Denny session)
- Gwin: agents evaluating agents (see notes, Industry Panel section)

INSTITUTIONAL CONTEXT

> "We have to pivot hard and immediately." — Titus Winters (Adobe)

9. Equity is a design problem, not an afterthought

> "Our students are your students. Your students might be our students." — Tasha Frankie (San Diego Mesa College)
>
> "Let us shadow you when you change your curriculum." — Tasha Frankie, on community colleges needing early communication about curriculum changes

Students don't arrive with equal access to AI tools, equal prior experience, or equal confidence in using them. If we redesign our curriculum around GenAI without intentionally designing for equity, we risk widening the gaps between students who already have advantages and those who don't. The students who most need the skills we're emphasizing (Principles 3-5) are often the ones with the least access to the tools that develop them.

Key equity questions for any curriculum redesign:

- **Access:** Paid vs. free tools. UW Madison's model: expect Claude Max subscription; cover it for students with financial need.
- **Literacy gaps:** 88.9% of students already use GenAI, but learned only through informal means. Formal instruction changes behavior.
- **Transfer alignment:** Community colleges face a 2-year implementation lag. If we change prerequisites, they need to know now. The same applies to BYU-I and BYU-H, which benefit from early communication about our curriculum direction so they can fast-follow.

- **Accessibility:** Not all tools are equally accessible (e.g., Gemini CLI currently more screen-reader accessible than Claude Code).

The Temple study found 100% of students now use ChatGPT, but the barriers are mostly cognitive and metacognitive (58.5% combined), not logistical. The Michigan GenAI contracts approach directly addresses metacognitive barriers by asking students to set their own goals and rules.

Key references:

- Equity Faculty Planner (workshop shared folder)
- Temple: barriers pie chart: cognitive 32.4%, metacognitive 26.1% (see notes, Temple session)
- Michigan GenAI Contracts
- Gorson Benario et al. (https://dl.acm.org/doi/10.1145/3641554.3701859)

10. The research loop itself is being automated

> "All LLM frontier labs will do this. It's the final boss battle." — Andrej Karpathy, on autoresearch

Note: This principle extends beyond the undergraduate focus to address graduate training and faculty research practices.

Our graduate students (and increasingly our undergraduates) will conduct research in a world where the experimental iteration loop is automated. The skills we need to develop in student researchers are shifting from "can you run this experiment?" to "which experiment is worth running, and how do you know the results are meaningful?" If we don't adjust how we train researchers, we produce graduates who can operate machinery but can't formulate the questions that make the machinery useful.

Karpathy's autoresearch (released March 7-8, 2026): a 630-line Python script that runs ~100 ML experiments overnight on a single GPU. What remains human: authoring the research strategy, choosing evaluation metrics that measure the real goal, knowing which hypotheses are worth testing. Publication norms and research credit are unsettled; informal norms are being set now through advising and reviewing decisions. This is urgent for CS faculty.

The uncomfortable implication for universities: the parts of education under the most economic pressure (mentorship, hands-on research, apprenticeship, fieldwork) are precisely the parts becoming more irreplaceable. Lecture-based coverage of automatable skills is worth less. Deep domain formation is worth more.

Key references:

- Gwin: agentic coding session (see notes, GitHub Agentic Coding section)

- Karpathy autoresearch: discussed in Gwin agentic coding session (see notes, GitHub Agentic Coding section)